# ЛР1. Подключение и разработка интерфейса на Angular

**Дисциплина:**«Протоколы и интерфейсы информационных систем».

## Легенда

ИТ-отдел учебного центра получает обращения о неисправных компьютерах, проекторах и сетевом оборудовании. Обращения теряются в переписке. Нужна информационная система, в которой можно зарегистрировать заявку, увидеть её статус и найти связанные сведения об оборудовании. Команда уже подготовила заготовку проекта: структуру, инструменты сборки, модель данных и тесты. Ваша задача — разобраться в чужом коде, подключить предусмотренные части и реализовать интерфейс. Это обычная ситуация в разработке: новый специалист начинает с существующего репозитория, а не с пустой папки.

## Что вы сделаете в лабораторной работе

- подготовите и запустите существующий проект;
- настроите маршрутизацию между страницами Angular;
- реализуете фильтрацию заявок;
- свяжете состояние Angular Signals с интерфейсом;
- выведете карточки заявок и счётчик;
- выполните отладку через DevTools;
- проверите решение автоматическими тестами.

## Входные знания

Массивы, объекты, функции, filter в *JavaScript*, базовые *HTML/CSS* и работа с каталогами в терминале.

## 1. Теория

### 1.1. От интерфейса к данным

Информационная система включает сведения, правила их обработки, программные компоненты и работу людей. В нашем случае сущности — оборудование и заявки; действия — зарегистрировать обращение, изменить статус, найти незакрытое обращение. Страница сайта — только часть системы.

Браузер отображает сведения и принимает действия пользователя. Сервер проверяет запросы и выполняет операции. База данных сохраняет согласованное состояние. Например, отсутствие

кнопки удаления в браузере само по себе не запрещает удаление: сервер обязан проверять допустимость операции.

> 💡 Схема взаимодействия
>
> Пользователь → Angular в браузере ↔️ HTTP/JSON ↔️ Flask ↔️ MySQL/SQL

HTTP — протокол обмена. JSON — формат представления данных. SQL — язык запросов. Это разные понятия. В ЛР №1 массив объектов заменяет будущий ответ API только для разработки интерфейса. Он не становится базой данных и не обеспечивает сохранение изменений.

*Node.js* выполняет инструменты *Angular* и сервер разработки. Код самого интерфейса после загрузки выполняется в браузере. Поэтому хранить пароль *MySQL* в *TypeScript*-файле нельзя: клиентский код получает пользователь. MDN: обзор HTTP.

> ! КОНТРОЛЬНЫЙ ВОПРОС
>
> Назовите компонент системы, который: показывает сообщение об ошибке; окончательно проверяет статус заявки; сохраняет заявку; собирает TypeScript. Почему одного браузера недостаточно для всего сервиса?

### 1.2. Репозиторий

*git clone* загружает существующий репозиторий. *npm ci* устанавливает зависимости по *lock-файлу. npm* start запускает команду проекта. Эти действия не взаимозаменяемы. *package.json* описывает зависимости и команды, а *package-lock.json* фиксирует разрешённое дерево пакетов. *Lock-файл* нужно сохранять в *Git; node_modules* — нет.

> ! КОНТРОЛЬНЫЙ ВОПРОС
>
> Почему последовательность git clone → npm ci → npm start подходит для этой работы, а ng new создаёт другую исходную точку? Что получит одногруппник, если отправить ему только src без конфигурации?

## 2. Подготовка проекта и клонирование репозитория

### 2.1. Подготовка инструментов

Установите *Git*, *Node.js 24.x*, редактор, *Chrome* или *Edge* и *Python 3.12* (допустим 3.10+). *Angular CLI* в проекте локальный и будет установлен через *npm ci*.

В терминале выполните:

```
git --version
node --version
```

```
npm --version
python --version
```

Ожидаются номера версий: *Node.js* — v24..... *Angular 21.x* совместим с *Node.js 24.x.* В этой работе CLI локальный, его устанавливает *npm ci.* Глобальная установка *CLI* не требуется. Не обновляйте пакеты до *latest* в ходе занятия. Таблица совместимости Angular.

В Windows при блокировке *npm.ps1* используйте терминал *cmd* или команды *npm.cmd* и *npx.cmd*. Не меняйте общесистемную политику выполнения ради запуска проекта.

## 2.2. Клонирование репозитория в GitHub

> ! ПРАКТИЧЕСКОЕ ЗАДАНИЕ — Получить рабочую копию проекта
>
> 1. Скопируйте URL своего репозитория.
> 2. Клонируйте проект в каталог it-service-desk.
> 3. Перейдите в каталог проекта.
> 4. Создайте рабочую ветку course.
>
> **Результат:** В терминале вы находитесь в корне it service-desk и работаете в ветке course.

В команде ниже замените *URL_ВАШЕГО_РЕПОЗИТОРИЯ* на скопированный адрес. Это обозначение, а не настоящий URL.

```
git clone URL_ВАШЕГО_РЕПОЗИТОРИЯ it-service-desk
cd it-service-desk
git switch -c course
```

## 2.3. Подготовка backend-окружения

Находясь в корне полученного *it-service-desk*, создайте Python-окружение:

```
python -m venv backend/.venv
```

На *macOS/Linux* используйте *python3 -m venv backend/.venv*, если команда python отсутствует. Активируйте окружение способом для своего терминала:

| Терминал | Команда из корня |
| --- | --- |
| Windows cmd | backend\.venv\Scripts\activate.bat |
| Windows PowerShell | .\backend\.venv\Scripts\Activate.ps1 |

| Терминал | Команда из корня |
| --- | --- |
| macOS/Linux bash или zsh | source backend/.venv/bin/activate |

Если *PowerShell* не разрешает активацию, используйте *cmd* или запускайте backend\.venv\Scripts\python.exe напрямую. Виртуальное окружение изолирует Python-пакеты, но не устанавливает *MySQL* и не заменяет npm.

```
Python: venv.
python -m pip install -r backend/requirements-dev.txt
python -c "import sys; print(sys.executable)"
cd backend
python -m pytest tests/test_starter.py -q
cd ..
```

Ожидается путь к *Python* внутри *backend/.venv* и одна успешная проверка. Если выводится системный
*Python*, активируйте окружение ещё раз. В новом терминале активацию повторяют; переустанавливать
окружение каждый раз не требуется.

Скопируйте *backend/.env.example* в *backend/.env* средствами редактора. Оставьте STORAGE=memory. Файл *.env* игнорируется *Git*. Значения DB пока не используются.

## 2.4. Подготовка и запуск frontend

> **!** ПРАКТИЧЕСКОЕ ЗАДАНИЕ — Установить зависимости Angular и запустить проект
>
> 1. Перейдите в frontend.
> 2. Установите зависимости командой npm ci.
> 3. Проверьте локальную версию Angular.
> 4. Запустите сервер разработки.
>
> **Результат:** Angular-приложение открывается в браузере; терминал не содержит ошибок сборки.

Теперь установите зависимости *Angular,* включая локальный *CLI*:

```
cd frontend
npm ci
npx ng version
npm start
```

Если вы воспользуетесь ZIP, то распакуйте его, откройте папку *it-service-desk* и выполните *git init*, затем *git switch -c course*; далее выполните создание Python-окружения, установку зависимостей backend и frontend по шагам выше.

## 2.5. Ориентирование в проекте

```
it-service-desk/
  frontend/
    src/app/
      app.ts, app.html, app.routes.ts
      models/ticket.ts
      data/demo-tickets.ts
      features/tickets/filter-tickets.ts
      pages/home/
      pages/tickets/
      pages/about/
      pages/not-found/
      services/
      starter.spec.ts
    src/acceptance/lab1.spec.ts
    package.json, package-lock.json, angular.json
  backend/                 будущий Flask
  database/                будущие SQL-сценарии
  e2e/                     будущие сквозные тесты
  docs/                    методичка, контракт, отчёт
  .vscode/                 запуск отладчика
  .github/workflows/       проверка на GitHub
```

## 2.6. Небольшая проверка backend без реализации API

Остановите *Angular* только при необходимости. В отдельном терминале активируйте *.venv*, перейдите в *backend* и выполните:

```
# it-service-desk/backend
python -m flask --app app:create_app run --debug --port 5000
```

Откройте *http://127.0.0.1:5000/api/health*. Ожидается *{"status":"ok","storage":"memory"}*.
Запрос */api/tickets* пока возвращает *501*. Это задание второй ЛР, а не ошибка установки.
В первой ЛР браузерный интерфейс к API ещё не подключается.

# 3. Практика создание интерфейса

## 3.1. Подключите страницы и меню

### Теория: компонент и маршрут

Компонент объединяет класс, *HTML-шаблон* и оформление. *RouterLink* выполняет навигацию внутри приложения, *RouterOutlet* задаёт место вывода страницы, *RouterLinkActive* отмечает текущий маршрут. *Wildcard* ** должен располагаться последним, иначе он перехватит остальные адреса..
Angular:
маршруты.

### Небольшой пример подключения

```
# frontend/src/app/app.routes.ts
import { Tickets } from './pages/tickets/tickets';
// Внутри массива routes, перед **:
{ path: 'tickets', component: Tickets },
```

Вторая строка — элемент существующего массива, а не отдельная инструкция верхнего уровня. Аналогично подключите *About*. Provider *provideRouter(routes)* уже есть в *app.config.ts*.

> **!** ПРАКТИЧЕСКОЕ ЗАДАНИЕ — Настроить маршрутизацию и меню
>
> 1. В frontend/src/app/app.routes.ts импортируйте Tickets и About из уже созданных файлов.
> 2. Добавьте маршруты tickets и about перед **. Сохраните home и перенаправление с пустого пути.
> 3. В frontend/src/app/app.ts добавьте RouterLink и RouterLinkActive в импорт из @angular/router и в imports компонента.
> 4. В frontend/src/app/app.html создайте nav с aria-label="Основная навигация".
> 5. Добавьте ссылки /home, /tickets, /about с подписями «Главная», «Заявки», «О системе».
> 6. Для активной ссылки используйте класс active и ariaCurrentWhenActive="page".
>
> **Результат:** Меню работает; /tickets и /about открывают соответствующие страницы; неизвестный адрес показывает «Страница не найдена».

Небольшой пример синтаксиса на другом маршруте:

```
# frontend/src/app/app.html
<a routerLink="/help" routerLinkActive="active"
   ariaCurrentWhenActive="page">Справка</a>
```

> 💡 Проверка
>
> Меню работает, */tickets* показывает заголовок списка, */about* — заголовок описания, неизвестный адрес — «Страница не найдена». При прямом обновлении */tickets* локальный сервер также открывает страницу.

> ! Контрольный вопрос
>
> Предскажите результат переноса ** в начало массива, проверьте и восстановите порядок. Наличие текста «Страница не найдена» внутри Angular само по себе не означает HTTP-ответ 404.

## 3.2. Реализуйте отбор заявок

### Теория: модель и контракт

Откройте *models/ticket.ts*. Заявка содержит *id, equipmentId, title, description, status*. Числовой *id* определяет запись; заголовок может меняться и не подходит как устойчивый идентификатор.

*TicketStatus* допускает только *'new' | 'in_progress' | 'closed'*. Такой тип помогает обнаружить опечатку в исходниках. TypeScript проверяет код во время разработки, но не валидирует автоматически произвольный JSON из сети. Внешние данные потребуют проверки на сервере. TypeScript: повседневные типы.

> ! Контрольный вопрос
>
> Объясните две ошибки: заявка с id: '4' и status: 'opened'. Почему отсутствие ошибки TypeScript ещё не доказывает корректность ответа внешнего API?

### Теория: чистая функция

Функция отбора получает данные и режим, возвращает подходящие записи. Она не должна менять исходные объекты, обращаться к *DOM* или устанавливать режим интерфейса. Такой небольшой участок логики легко проверять отдельно от *Angular*.

*filter* создаёт новый массив подходящих элементов. Выражение *item.status !== 'closed'* сохраняет и новые заявки, и заявки в работе. «Незакрытые» не означает «только новые».

### Задание

> ! ПРАКТИЧЕСКОЕ ЗАДАНИЕ - Реализовать функцию filterTickets
>
> 1. Откройте frontend/src/app/features/tickets/filter-tickets.ts.
> 2. Замените заглушку filterTickets, сохранив сигнатуру, TicketFilter и имена экспортов.
> 3. Для режима all верните все заявки.
> 4. Для open оставьте заявки, у которых status !== 'closed'.

5. Для new оставьте заявки со status === 'new'.
6. Убедитесь, что пустой вход возвращает пустой результат и исходный массив не изменяется.

**Результат:** На базовых данных: all → ID 1,2,3; open → 1,2; new → 1.

В *src/app/features/tickets/filter-tickets.ts* замените заглушку функции *filterTickets*. Сигнатуру, тип *TicketFilter* и имена экспортов сохраните.

| Режим | Условие | ID на базовых данных |
|---|---|---|
| all | Все заявки | 1, 2, 3 |
| open | Статус неclosed | 1, 2 |
| new | Статус равенnew | 1 |

Требования: исходный массив не меняется; пустой вход возвращает пустой результат; решение работает с любыми ID и названиями, а не только с тремя примерами. Не подставляйте заранее массивы [1, 2] вместо проверки статуса.

Пример одной ветви, которую нужно поместить внутрь *filterTickets* вместо заглушки:

```
# frontend/src/app/features/tickets/filter-tickets.ts
if (mode === 'all') return [...items];
```

Для *open/new* реализуйте условие статуса. Не оставляйте финальное return [] для всех режимов.

### 💡 Подсказки

1. Начните с отдельной ветви для all.
2. Для остальных режимов используйте filter.
3. Режимы интерфейса и статусы модели — разные типы, хотя значение 'new' встречается в обоих.

### ! Контрольный вопрос

Если в тестовом наборе изменить статус заявки №2 на closed, какие ID получатся для all, open и new? Сначала сделайте прогноз, затем проверьте.

## 3.3. Соедините состояние и шаблон

### Теория: сигналы

Состояние — значения, меняющиеся во время работы интерфейса. Сигнал хранит значение и позволяет *Angular* отслеживать его использование. В заготовке *filter* содержит текущий режим, а *tickets* — массив.

Чтение выполняется вызовом *filter()*, запись — *filter.set(...)*. *computed* вычисляет производный список из текущих сигналов. Эта связка уже подготовлена в *tickets.ts:* вам нужно изменить режим и правильно показать результат. Состояние в памяти не сохраняется автоматически после обновления страницы. Angular: Signals.

### Теория: шаблон

Интерполяция *{{ value }}* выводит значение как текст. *(click)="handler()"* вызывает обработчик. *[attr.aria-pressed]="condition"* отражает состояние кнопки для средств доступности. Не заменяйте обычные кнопки кликабельными *div* — у кнопок уже есть клавиатурное управление. Angular: привязки.

*@for* повторяет фрагмент для коллекции; *track ticket.id* связывает элемент с устойчивым идентификатором. *@empty* показывает альтернативу для пустого результата. Эти конструкции управляют отображением, не выполняя запись в БД. Angular: управляющие конструкции.

> **!** ПРАКТИЧЕСКОЕ ЗАДАНИЕ - Реализовать обработчик фильтра
>
> 1. Откройте frontend/src/app/pages/tickets/tickets.ts.
> 2. Реализуйте setFilter(mode).
> 3. Запишите выбранный режим в существующий сигнал filter через this.filter.set(mode).
> 4. Не меняйте статусы заявок и не создавайте три независимых boolean-флага.
>
> **Результат:** Нажатие на кнопку сможет менять единый режим фильтра.

> **!** ПРАКТИЧЕСКОЕ ЗАДАНИЕ - Добавить кнопки фильтра
>
> 1. Откройте frontend/src/app/pages/tickets/tickets.html.
> 2. Добавьте три кнопки type="button": «Все», «Незакрытые», «Только новые».
> 3. Передавайте в setFilter значения 'all', 'open', 'new'.
> 4. Используйте data-testid: filter-all, filter-open, filter-new.
> 5. Привяжите aria-pressed к сравнению filter() с режимом соответствующей кнопки.
>
> **Результат:** Активная кнопка отражает текущий режим, а нажатие меняет список.

| Подпись | Аргумент обработчика | data-testid |
|---|---|---|
| Все | 'all' | filter-all |
| Незакрытые | 'open' | filter-open |
| Только новые | 'new' | filter-new |

Привяжите *aria-pressed* к сравнению текущего режима с режимом кнопки. Например, состояние кнопки для режима *'new'* определяется выражением *filter() === 'new'*. Сохраните единый источник

состояния в компоненте.

Пример одной кнопки; остальные сделайте по таблице:

```
# frontend/src/app/pages/tickets/tickets.html
<button type="button" data-testid="filter-new"
  [attr.aria-pressed]="filter() === 'new'"
  (click)="setFilter('new')">Только новые</button>
```

В *setFilter(mode)* используется *this.filter.set(mode)*. *Readonly* у поля запрещает заменить сам сигнал другим объектом, но не запрещает установить его новое значение.

> ! ПРАКТИЧЕСКОЕ ЗАДАНИЕ - Вывести карточки заявок
>
> 1. В том же tickets.html выведите visibleTickets() через @for.
> 2. Для каждой карточки задайте data-testid="ticket-card" и [attr.data-ticket-id]="ticket.id".
> 3. Используйте track ticket.id.
> 4. Покажите title, description, номер заявки, equipmentId и русскую подпись statusLabels[ticket.status].
> 5. Счётчик ticket-count должен показывать длину visibleTickets().
> 6. В @empty добавьте data-testid="empty-state" и текст «По выбранному условию заявок нет.».
>
> **Результат:** При переключении фильтра меняются карточки и счётчик; пустой результат отображается отдельным сообщением.

Основа цикла в существующем *<ul>*:

```
# rontend/src/app/pages/tickets/tickets.html
@for (ticket of visibleTickets(); track ticket.id) {
  <li class="ticket" data-testid="ticket-card" [attr.data-ticket-id]="ticket.id">
    <h3>{{ ticket.title }}</h3>
    <!-- Добавьте остальные поля из задания. -->
  </li>
} @empty {
  <li data-testid="empty-state">По выбранному условию заявок нет.</li>
}
```

*data-testid* и *data-ticket-id* — договорённость учебных проверок. Они не задают оформление и не являются бизнес-данными. Сохраняйте эти имена, чтобы тест мог обратиться к нужному элементу.

## 4. Отладка и самостоятельная проверка

### 4.1 DevTools и breakpoint

Отладчик останавливает программу на выбранной строке и позволяет увидеть значения во время выполнения. Рабочая последовательность: воспроизвести симптом, сформулировать гипотезу, проверить состояние, исправить причину и повторить сценарий. Chrome DevTools: отладка JavaScript.

> ! ПРАКТИЧЕСКОЕ ЗАДАНИЕ - Найти ошибку с помощью breakpoint
>
> 1. После правильной реализации временно измените setFilter так, чтобы он всегда устанавливал 'new'.
> 2. Откройте страницу заявок и нажмите «Незакрытые». Сравните ожидаемый и фактический результат.
> 3. В DevTools → Sources поставьте breakpoint внутри setFilter.
> 4. Нажмите «Все» и проверьте mode и this.filter() до записи.
> 5. Выполните Step over и снова проверьте значение.
> 6. Исправьте ошибку и повторите open → new → all.
>
> **Результат:** Вы можете объяснить, почему интерфейс показывал неправильный режим, и подтвердить исправление.

> ! ПРАКТИЧЕСКОЕ ЗАДАНИЕ - Сравнить загрузку документа и SPA-навигацию
>
> 1. Откройте DevTools → Network и выберите фильтр Document.
> 2. Обновите страницу.
> 3. Затем перейдите на другую страницу через меню RouterLink.
> 4. Сравните сетевые запросы.
>
> **Результат:** Вы видите загрузку документа при обновлении и отсутствие повторной загрузки всего документа при обычной SPA-навигации.

## 4.2 Автоматическая проверка

Vitest запускает проверки, TestBed создаёт компоненты, RouterTestingHarness проверяет маршруты. Приёмочные тесты читают реальные элементы шаблона и нажимают кнопки. Тест только на создание компонента не обнаружил бы фильтр с неверным поведением. Angular: тестирование.

В пакете предусмотрены:

```
# it-service-desk/frontend
npm test: исправность заготовки и ваши тесты вне acceptance;
npm run test:lab1: 14 открытых требований первой работы;
```

```
npm run test:all: обе группы вместе;
npm run check:lab1: сборка, затем все тесты.
```

> 💡 Готовность основной части
>
> **Готовность основной части**
> Работа считается технически готовой, когда npm run check:lab1 выполняется без ошибок.
> Если npm test зелёный, но acceptance-тесты не запускались, лабораторная ещё не считается полностью проверенной.

# 5. Затруднения и способы проверки

| Не найдены node/npm/git | Проверьте установку, PATH и откройте новый терминал. |
|---|---|
| npm.ps1 заблокирован | Используйте cmd или npm.cmd / npx.cmd. |
| Команда не видит package.json | Перейдите в frontend. |
| Страница пустая | Проверьте Console, терминал, RouterOutlet и маршруты. |
| /tickets показывает «не найдена» | Проверьте импорт страницы и порядок маршрутов. |
| Нажатие не меняет список | Проверьте setFilter, сигнал filter и filterTickets. |
| Счётчик всегда 3 | Используйте visibleTickets().length, а не tickets().length. |
| Тест не находит элемент | Проверьте точные data-testid и data-ticket-id. |
| Порт 4200 занят | Остановите старый сервер или используйте npm start -- --port 4201. |

# 6. Дополнительное задание

> ! ПРАКТИЧЕСКОЕ ЗАДАНИЕ - Добавить поиск по заголовку
>
> 1. Добавьте строку поиска по title без учёта регистра.
> 2. Перед сравнением удаляйте пробелы по краям строки.
> 3. Поиск должен работать совместно с выбранным режимом all/open/new.
> 4. При пустом поиске базовые acceptance-тесты должны продолжать работать.
> 5. Добавьте тесты для найденного и пустого результата.
>
> **Результат:** Фильтр по статусу и поиск работают совместно. Дополнительное задание выполняется после обязательной части.

# 7. Что сдавать

> 💡 Перечень
>
> 1. Проект с выполненными изменениями в рабочей ветке.

2. Рабочий интерфейс с маршрутами, фильтрами, карточками и счётчиком.
3. Успешный результат npm run check:lab1.
4. Ответы на контрольные вопросы.
5. Дополнительное задание.

## 8. Материалы для подготовки

| Материалы | Что повторить |
| --- | --- |
| GitHub | Создание личной копии и работа с репозиторием. |
| Angular: совместимость | Совместимость Angular 21.x и Node.js. |
| Angular: Learn Angular | Компоненты, шаблоны и события. |
| TypeScript: Everyday Types | Объекты и объединения литералов. |
| Angular: компоненты | Метаданные и imports. |
| Angular: маршруты | Порядок путей и wildcard. |
| Angular: Signals | Сигналы и computed. |
| Angular: шаблоны | @for, track и @empty. |
| Chrome DevTools | Breakpoint и пошаговое выполнение. |
| Angular: тесты | Тестовая среда и запуск. |
| MDN: HTTP | Клиент, сервер, запрос и отв |